



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

TÉRKÉPTUDOMÁNYI ÉS GEOINFORMATIKAI INTÉZET

Ellenfél alapú támadások (FGSM, PGD)
implementációja
konvolúciós neurális hálózatokhoz
interaktív tesztkörnyezetben

Szerző:

Gabrity Gábor

Programtervező Informatikus BSc

Belső témavezető:

Varga Zsófia

adjunktus

Külső témavezető:

Szabó András Miklós

tanársegéd

Budapest, 2026

Nyilatkozat

Alulírott Gabrity Gábor, az Eötvös Loránd Tudományegyetem Informatikai Karának hallgatója kijelentem, hogy ezt a szakdolgozatot saját magam készítettem, és abban csak a megadott forrásokat használtam fel. Minden olyan részt, amelyet szó szerint vagy azonos tartalommal, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

A dolgozat bírálója által hátrányos értékeléssel nem járó exkluzív elektronikus hozzáférést a benyújtástól számított két évig nem kérem.

Budapest, 2026. szeptember 20.

Gabrity Gábor s.k.
hallgató aláírása

Tartalomjegyzék

Rövidítés- és jelölésjegyzék	4
1. Bevezetés	5
1.1. Motiváció és a témaválasztás indoklása	5
1.2. A megoldandó feladat rövid, közérthető leírása	5
1.3. A dolgozat célja és hozzájárulásai	5
1.4. A dolgozat felépítése	5
2. Felhasználói dokumentáció	6
2.1. A megoldott probléma rövid megfogalmazása	6
2.2. A felhasznált módszerek rövid áttekintése	6
2.3. Rendszerkövetelmények	6
2.4. Telepítés és első indítás	6
2.5. A grafikus felhasználói felület bemutatása	6
2.6. Tipikus felhasználási forgatókönyvek	6
2.7. Hibaüzenetek és gyakori problémák	6
3. Fejlesztői dokumentáció	7
3.1. A probléma részletes specifikációja	7
3.1.1. A modell, a bemenet és a címke	7
3.1.2. A támadás célja	8
3.1.3. Targeted és non-targeted mód	8
3.1.4. A white-box hozzáférési feltétel	8
3.1.5. A szoftver célja és beállítható paraméterei	8
3.1.6. A felhasználói funkciók vázlatos áttekintése	8
3.2. Elméleti háttér és a használt fogalmak definíciója	9
3.2.1. Az osztályozó modell és a logitok	9
3.2.2. Valószínűségek és a modell predikciója	9

3.2.3.	Veszteségfüggvény és bemeneti gradiens	10
3.2.4.	Perturbáció és normakorlát	10
3.3.	Ellenfél alapú támadások áttekintése	11
3.4.	FGSM (Fast Gradient Sign Method)	11
3.4.1.	Célzott változat	11
3.4.2.	Előnyök és korlátok	12
3.5.	PGD (Projected Gradient Descent)	12
3.5.1.	Célzott változat	12
3.5.2.	Véletlen kiindulási pont	13
3.5.3.	Az α és T paraméterek megválasztása	13
3.5.4.	Előnyök és korlátok	13
3.6.	Továbbmutató elméleti vonatkozások	13
3.6.1.	Transzferálhatóság	14
3.6.2.	Adversarial training	14
3.6.3.	Linearitás	15
3.6.4.	Kapcsolódó munkák és további támadási módszerek	15
3.7.	Tervezési döntések és indoklásuk	15
3.8.	Védekezési stratégiák áttekintése	15
3.9.	A rendszer architektúrája	15
3.10.	A felhasznált technológiák	15
3.11.	Az egyes komponensek megvalósítási részletei	15
4.	Tesztelés	16
4.1.	Egységtesztek	16
4.2.	Integrációs tesztek	20
4.2.1.	GUI és támadási motor összekötése	20
4.3.	Automatizált GUI tesztek	20
4.4.	Funkcionális tesztek és felhasználói validáció	21
4.5.	Teszteredmények és értelmezésük	23
4.6.	A tesztelés korlátai	23
5.	Összefoglalás	24
5.1.	Elért eredmények	24
5.2.	Következtetések	24
5.3.	Továbbfejlesztési lehetőségek	24

Rövidítés- és jelölésjegyzék

1. fejezet

Bevezetés

- 1.1. Motiváció és a témaválasztás indoklása
- 1.2. A megoldandó feladat rövid, közérthető leírása
- 1.3. A dolgozat célja és hozzájárulásai
- 1.4. A dolgozat felépítése

2. fejezet

Felhasználói dokumentáció

2.1. A megoldott probléma rövid megfogalmazása

2.2. A felhasznált módszerek rövid áttekintése

2.3. Rendszerkövetelmények

2.4. Telepítés és első indítás

2.5. A grafikus felhasználói felület bemutatása

2.6. Tipikus felhasználási forgatókönyvek

2.7. Hibaüzenetek és gyakori problémák

3. fejezet

Fejlesztői dokumentáció

Ebben a fejezetben a megvalósítandó szoftver célját és funkcióit formális, matematikai keretben definiálom, mivel a későbbi fejezetekben tárgyalt algoritmusok (FGSM, PGD) és a védekezési modul is ugyanerre a jelölésrendszerre épülnek majd. Először a probléma szereplőit, a modellt, a bemenetet és a hozzá tartozó címkét vezetem be, majd ezek segítségével pontosan megfogalmazom, mit is jelent az ellenfél alapú perturbáció. Ezután az itt bevezetett jelölésrendszerre épülve sorra bemutatom a két megvalósított támadási algoritmust (FGSM, PGD), a mögöttük álló tervezési döntéseket, a kapcsolódó munkákat, a védekezési stratégiákat, végül a szoftver architektúráját, a felhasznált technológiákat és az egyes komponensek megvalósítását.

3.1. A probléma részletes specifikációja

3.1.1. A modell, a bemenet és a címke

Legyen adott egy f_θ neurális háló alapú osztályozó modell, melynek x egy bemenete és y annak helyes címkéje. Az x bemenet ebben az esetben egy kép pixeleinek valamely három színű kódolása, vektoriális formában, az y pedig a kép helyes osztályának indexe, például $y = 42$ az agámák osztályát reprezentálja. Amennyiben a képhez tartozó y helyes címke nem ismert, úgy a modell által meghatározott legvalószínűbb címkét $\hat{y}$ -nal jelöljük, és ezt használjuk referenciacímkeként.

3.1.2. A támadás célja

Célunk az x képre egy x_{adv} bemenet előállítása, mely az eredeti x -től szemmel csak alig látható perturbációban különbözik, viszont a modell által adott $\hat{y}_{\text{adv}} = \arg \max_i f_{\theta}(x_{\text{adv}})_i$ predikció a referenciacímktől eltér, vagy a klasszifikáció elbizonytalanodik.

3.1.3. Targeted és non-targeted mód

Alapvetően két támadási módot különböztetünk meg, az ún. targeted és non-targeted módot. Non-targeted esetben a támadás célja minél jobban elbizonytalanítani a helyes osztályozást, tehát hogy agáma helyett bármi mást mutasson. Targeted módban azt szeretnénk, hogy egy kijelölt y_{target} osztály irányába változzon a képünk (pl. a komodói varánusz osztálya felé).

3.1.4. A white-box hozzáférési feltétel

A támadás feltétele, hogy az f_{θ} modell gradienseihez és paramétereire teljes hozzáférésünk legyen. Ezt white-box hozzáférésnek nevezzük az AI red teaming kontextusában.

3.1.5. A szoftver célja és beállítható paraméterei

E szoftver célja a fenti ellenfél alapú támadó tesztkörnyezet szoftveres megvalósítása, különböző támadási algoritmusokkal (FGSM, PGD). A szoftver a PyTorch (torchvision) előtanított modelljeire épül, mely architektúra tetszőleges torchvision-moddal bővíthető. Jelenleg kettő van integrálva (ResNet50, MobileNetV2). Beállíthatóak a futás paraméterei:

- modell: ResNet50 vagy MobileNetV2, amelyen a támadást végrehajtjuk,
- ϵ , mely meghatározza a perturbáció maximális mértékét, tehát azt, hogy egy pixelen legfeljebb mekkora változást tapasztalhatunk, azaz a változás felső határát,
- célosztály, y_{target} , ha a mód targeted,
- iterációszám (PGD esetén hány lépésig engedjük futtatni az algoritmust).

3.1.6. A felhasználói funkciók vázlatos áttekintése

A fenti specifikációt a szoftver az alábbi, a felhasználó számára közvetlenül elérhető funkciókkal valósítja meg:

- a fájlrendszerről kiválasztott képet legördülő menüből kiválasztott modellel klasszifikáljuk,
- támadási algoritmus és paraméterek meghatározásával perturbációt és adversarial képet generálunk,
- megjelenítjük az eredeti képet, a perturbációt és az adversarial képet, továbbá mindkét (eredeti és adversarial) képet osztályozzuk is,
- a védekezési modulban JPEG-tömörítést vagy más képfeldolgozási szűrőt alkalmazhatunk az adversarial perturbáció hatásának utólagos csökkentésére vagy semlegesítésére.

3.2. Elméleti háttér és a használt fogalmak definíciója

3.2.1. Az osztályozó modell és a logitok

Jelöljük $f_\theta(x) = z$ -vel a modell outputjaként előállt valós számok tömbjét, ahol $z \in \mathbb{R}^C$ és C az osztályok száma (ImageNet esetén $C = 1000$). Ezt logitvektornak is nevezzük, mely elnevezés a logisztikus regresszióból származik, ahol a $\text{logit}(p) = \ln \frac{p}{1-p}$ függvény egy $p \in (0, 1)$ valószínűséget képez le a valós számegyenesre. A z_i komponensek páronkénti különbsége éppen az i -edik és j -edik osztály esélyhányadosának logaritmusával egyezik meg, $z_i - z_j = \ln \frac{p_i}{p_j}$, innen származik az elnevezés.

3.2.2. Valószínűségek és a modell predikciója

Mivel a z logitok önmagukban nem valószínűségek, a softmax függvénnyel alakítjuk át őket valószínűségi eloszlássá:

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad i = 1, \dots, C.$$

Tehát p_i a modell szerint annak a valószínűsége, hogy x az i -edik osztályba tartozik. A modell által meghatározott legvalószínűbb osztály indexét a legnagyobb valószínűségű komponens alapján kapjuk:

$$\hat{y} = \arg \max_i p_i.$$

3.2.3. Veszteségfüggvény és bemeneti gradiens

A $J(\theta, x, y)$ veszteségfüggvény a modell predikciója és a megadott helyes címke közötti eltérést méri. Esetünkben ez a keresztentrópia-veszteség

$$J(\theta, x, y) = -\log \frac{e^{z_y}}{\sum_{i=1}^C e^{z_i}},$$

ahol $z = f_\theta(x)$ a logitvektor és y a helyes osztály indexe. A veszteségfüggvény súlyok szerinti gradiense, $\nabla_\theta J(\theta, x, y)$, az a gradiensvektor, amelyet a backpropagation során a tanítás közben használunk a súlyok módosítására. Itt a gradiensszámítást egy további lépéssel a bemeneti réteig visszük – azaz a backpropagation folyamatát a hálón teljesen végigvezetjük –, és megkapjuk a bemeneti vektorra vonatkozó érzékenységet, $\nabla_x J(\theta, x, y)$. Ez a mennyiség azt mutatja meg, hogy egy adott bemeneti pixel vagy jellemző változása milyen mértékben befolyásolja a veszteséget, ezért ennek a mennyiségnek központi szerepe lesz a későbbiekben bemutatott adversarial-példa-generáló algoritmusok leírásában.

3.2.4. Perturbáció és normakorlát

Az adversarial képek definíciójánál szokás a perturbáció nagyságát valamely ℓ_p normával jellemezni; ebben a dolgozatban a ℓ_∞ normát használjuk. Egy perturbált kép általában az

$$x_{\text{adv}} = x + \delta$$

formában írható le, ahol δ a zaj, vagyis a képpontokban okozott változás. A ℓ_∞ normát a

$$\|\delta\|_\infty = \max_i |\delta_i|$$

kifejezés definiálja, azaz a legnagyobb abszolút értékű komponens adja a norma értékét. A perturbáció nagyságát ezzel a normával korlátozzuk:

$$\|\delta\|_\infty \leq \epsilon.$$

Ez esetünkben nagyon kézenfekvő, mivel a bemeneti kép minden pixelén (minden δ_i komponensen) külön-külön korlátozza a megengedett változást.

3.3. Ellenfél alapú támadások áttekintése

A mély neurális hálók, köztük a képfelismerő modellek is, különösen érzékenyek a bemeneti adatok kicsi, ám célzott módosításaira [1]. Az ilyen módosításokat ellenfél alapú perturbációknak nevezzük. Ezeknek a támadásoknak két algoritmusát fogjuk itt átvenni, melyek a taxonómiában a korábban említett white-box támadások közé tartoznak. A következő fejezetekben egy egylépéses ellenfél alapú támadást mutatunk be (FGSM), utána erre építve egy többlépéses algoritmust (PGD).

3.4. FGSM (Fast Gradient Sign Method)

Az FGSM algoritmust Goodfellow és szerzőtársai a Google-nél végzett kutatásaik alapján javasolták [1]. Az előző fejezetben megjegyeztem, hogy a $\nabla_x J(\theta, x, y)$ veszteségfüggvény gradiense az input értékre való érzékenységet jelenti. Természetes ötlet, hogy a bemenetet a gradienssel ellentétes irányba mozdítsuk el, ami várhatóan növeli a veszteségfüggvény értékét, azaz rontja a helyes osztályra adott predikciót. Ehhez feltesszük, hogy a modell az x pont kellően kis környezetében közelítőleg lineárisan viselkedik – ez Goodfellow és társai linearitási elmélete, amely szerint a modern neurális hálók, részben a szakaszonként lineáris aktivációk (pl. ReLU) miatt, lokálisan jól közelíthetők egy lineáris függvénnyel, még ha globálisan erősen nemlineárisak is [1]. Az FGSM algoritmusban az elmozdítást egyszerűen csak az előjel-függvény (sign) ϵ -szorosával tesszük meg, tehát nem vesszük figyelembe a gradiens nagysága által adott érzékenységet, csak az irányát. Így módon az $\epsilon \cdot \text{sign}(\cdot)$ minden egyes x_i komponensre kihasználja a teljes ϵ költségvetésünket. Tehát az FGSM adversarial példát a következő módon kapjuk meg:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)).$$

3.4.1. Célzott változat

Az alapváltozat a megadott helyes osztály veszteségét igyekszik maximalizálni, tehát minél inkább mást mutasson, mint ami a képen van. A célzott változatnál egy y_{target} célosztályt adunk meg, és azt szeretnénk, hogy a modell ebbe az osztályba sorolja az adversarial képet.

$$x_{\text{adv}} = x - \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y_{\text{target}})).$$

Tehát a képlet csak annyiban változik, hogy távolodás helyett az y_{target} -hoz tartozó veszteség minimuma felé közeledni szeretnénk, és y helyett y_{target} -ot használjuk.

3.4.2. Előnyök és korlátok

Mivel a gradienst a tanításban is használt backpropagation segítségével hatékonyan tudjuk kiszámolni, ezért a támadás gyors. Hátránya, hogy csak egyetlen, durva lépést tesz a megfelelő irányba, ezért – főleg az iteratív módszerekhez, például a PGD-hez képest – kevésbé hatékonyan képes megtéveszteni az osztályozót.

3.5. PGD (Projected Gradient Descent)

Ezt a módszert Madry és társai vezették be [2], és lényegében az FGSM egy lépéses ötletének iteratív általánosítása. Nem egyetlen nagy lépést teszünk a gradiens előjele irányába, hanem ugyanabba az irányba több, kisebb lépést hajtunk végre egymás után. A motiváció emögött az, hogy az FGSM linearitási feltevése csak x kellően kis környezetében állja meg a helyét, ezért előnyösebb, ha T darab kisebb, α nagyságú lépéssel dolgozunk, és a gradienst minden lépésben újraszámoljuk. Legyen $S = \{\delta : \|\delta\|_\infty \leq \epsilon\}$ a megengedett perturbációk halmaza; ekkor $x + S$ adja meg az x körüli, ϵ sugarú gömbbe eső, engedélyezett bemenetek halmazát. (ℓ_∞ norma esetén ez geometriailag valójában egy kocka, de általános ℓ_p értelemben vett gömbként hivatkozunk rá.) Mivel az iteráció túlléphet $x + S$ korlátján, az algoritmus minden lépés után egy projekcióval visszavetíti az eredményt erre a halmazra. Ez FGSM esetén a sign függvény miatt automatikusan teljesült, itt viszont T és α megválasztásától függően külön biztosítani kell. Az algoritmus formális definíciója tehát

$$x^{t+1} = \Pi_{x+S}(x^t + \alpha \cdot \text{sign}(\nabla_x J(\theta, x^t, y))), \quad x^0 = x,$$

ahol Π_{x+S} az $x + S$ halmazra való projekció.

3.5.1. Célzott változat

Az FGSM-hez hasonlóan itt is definiálunk egy célzott változatot, amely az ottanival analóg módon adható meg:

$$x^{t+1} = \Pi_{x+S}(x^t - \alpha \cdot \text{sign}(\nabla_x J(\theta, x^t, y_{\text{target}}))).$$

3.5.2. Véletlen kiindulási pont

Madry és társai kísérletileg mutatták meg, hogy a veszteségfelszín lokális maximumai jól koncentráltak, így egy véletlen kezdőpontból indított PGD is megbízhatóan jó megoldást talál [2]. Ennek a modellek robusztusságának tesztelésében is van szerepe: mivel a veszteségfelszínnek több, egymástól távol eső lokális maximuma is lehet, egy fix (pl. magából x -ből induló) kezdőpont esetén a szerencsének nagy szerepe lehetne, és az eredmény nem feltétlenül mutatná a modell valódi sebezhetőségét. Ezért a véletlen kezdőpontot az ϵ sugarú gömbön egyenletes eloszlással választjuk meg:

$$x^0 = \Pi_{x+S}(x + \eta), \quad \eta \sim \mathcal{U}(-\epsilon, \epsilon)^d.$$

3.5.3. Az α és T paraméterek megválasztása

A PGD viselkedését – az ϵ perturbációs korláton felül – két további paraméter határozza meg. T az iterációk számát jelöli, ez adja meg az algoritmus számítási költségét, ugyanakkor több iteráció jellemzően erősebb támadást eredményez. Az α lépésmagyságot úgy kell megválasztani, hogy túl kicsi érték esetén lassú a konvergencia, túl nagy érték esetén pedig instabilitáshoz vezethet. Tipikus választás: $\alpha = \epsilon/T$ vagy $\alpha = 2,5 \cdot \epsilon/T$.

3.5.4. Előnyök és korlátok

A PGD az FGSM-nél lényegesen erősebb támadásokat képes generálni, mivel több kis lépésben közelíti a lokális optimumot. A véletlen kezdőponttal kombinálva különösen hatékony a robusztusság kiértékelésében. Hátránya, hogy T -szer annyi gradiens-számítást igényel, mint az FGSM, így nagyobb a számítási költsége.

3.6. Továbbmutató elméleti vonatkozások

Ebben a fejezetben azokat az elméleti vonatkozásokat foglalom össze, amelyek nem képezik szigorúan a dolgozatban megválasztott szoftver részét, ám fontos kiegészítői annak.

3.6.1. Transzferálhatóság

3.6.2. Adversarial training

Ezeknek a támadási algoritmusoknak a szerepe nem csak a modell megtévesztése, hanem felhasználhatók a tanítás során a modell robusztusabbá tételére is. Ennek lényege, hogy a tiszta x kép mellett a hozzá tartozó x_{adv} adversarial képet is bevonjuk a veszteségfüggvénybe, és a modell paramétereit az így kapott kombinált célfüggvényre optimalizáljuk.

Goodfellow és társai az FGSM-mel előállított x_{adv} perturbációra a következő kombinált veszteségfüggvényt javasolták [1]:

$$\tilde{J}(\theta, x, y) = \lambda J(\theta, x, y) + (1 - \lambda) J(\theta, x_{\text{adv}}, y), \quad \lambda = 0,5.$$

Az új veszteségfüggvény tehát a tiszta és az adversarial minta hibáját egyaránt bünteti, még hozzá úgy, hogy x_{adv} -ot minden tanítási lépésben a modell aktuális θ paramétereivel újraszámolják. Így tehát a modell a tanítás során folyamatosan fejlődik, és az aktuális állapota által generált adversarial példákon tanul tovább dinamikusán, nem pedig egy előre legyártott, statikus adversarial halmazon. Kísérleteik megmutatták, hogy ez egy hatékony módszer: az FGSM-alapú adversarial példákon mért hibaarány 89,4%-ról 17,9%-ra csökkent.

Madry és társai megmutatták, hogy PGD-vel is alkalmazható az adversarial training, és erősebb védekezést eredményez [2]. Itt azonban a tiszta és adversarial veszteséget kombináló FGSM-formulával ellentétben nincs szó keverésről: a tanítás során a tiszta x kép helyett kizárólag a hozzá tartozó, PGD-vel előállított x_{adv} -on számolják a veszteséget, tehát a modell csak az adversarial bemeneteken tanul. Mivel a PGD erősebb támadás, mint az FGSM, az ellene edzett modell a gyengébb, egylépéses támadásokkal szemben is ellenállóbb marad, míg a fordítottja nem feltétlenül igaz.

3.6.3. Linearitás

3.6.4. Kapcsolódó munkák és további támadási módszerek

3.7. Tervezési döntések és indoklásuk

3.8. Védekezési stratégiák áttekintése

3.9. A rendszer architektúrája

3.10. A felhasznált technológiák

3.11. Az egyes komponensek megvalósítási részletei

4. fejezet

Tesztelés

4.1. Egységtesztek

A program tesztkészletének felépítésénél a szakirodalomból ismert *tesztpiramisnak* nevezett hierarchiát követtem [3]. Ennek alapját a gyorsabb és kevés komponenst érintő tesztek, az egységtesztek képezik, melyek a lehető legkisebb elkülöníthető egységek tesztelését valósítják meg. Futási idejük jellemzően ezredmásodperces tartományban mozog és akár lefedhetik az input paraméterek teljes értékű készletét. Az elkészítésüknél fontos szempont a többi egységtől való függetlenség (izoláció) és az perifériáktól való függetlenség, I/O, adatbázis, hálózat, stb. Ezeknek a teszteknek az elkészítésére érdemes nagy figyelmet fordítani, mivel a korai szakaszban feltárt hibák javítása általában lényegesen olcsóbb, mint a későbbi, integráció vagy éles környezetben felfedezetté.

Az egységtesztek tudatosan nem a valódi, előtanított ResNet50/MobileNetV2 modelleket használják. Az `attack_engine` tesztknél egy `tiny_model` fixture ad egy 5 osztályos, 8×8 pixeles bemenetet feldolgozó, egyetlen lineáris rétegből álló hálót. Ez a döntés a tesztek sebességét (a valódi háló letöltése és felépítése másodperces nagyságrendű lenne) és izoláltságát szolgálja: a teszt kizárólag a saját kódunk logikáját vizsgálja, nem a harmadik féltől származó architektúra pontosságát.

Minden véletlent felhasználó fixture (pl. a bemeneti képet generáló `sample_image_tensor`) maga hívja meg a `torch.manual_seed`-et közvetlenül az adat előállítás előtt, így a determinizmus független a fixture-ök végrehajtási sorrendjétől. Az egységtesztek szintjén nem támadási sikerrátát mérünk (ehhez sok mintás, statisztikai kiértékelés kellene), hanem az algoritmusok tulajdonságait ellenőrizzük konkrét, kis méretű bemeneteken.

Ebben a részben a számosságuk miatt nem mutatom be az összes egységtesztet, csak a középpontban álló `attack_engine.py` modult mivel ez valósítja meg az alkalmazás lényegi logikáját (az FGSM és PGD algoritmusokat, valamint az ezeket kiszolgáló `predict()` és `attack()` függvényeket).

A `predict()` és a diszpécser (`attack()`) tesztjei

`test_predict_returns_topk_shapes`: Ellenőrzi, hogy `topk=3` esetén a visszaadott valószínűség- és index-tenzor alakja $(1, 3)$, azaz a batch- és k -dimenzió megfelel a kérésnek.

`test_predict_returns_valid_distribution`: A `topk=5` valószínűségek mindegyike a $[0, 1]$ intervallumba esik, és – mivel az 5 osztályos `tiny_model` minden osztálya lekérdezésre kerül – összegük 1,0-ra `allclose`. Ez a softmax kimenet valódi valószínűségi eloszlás voltát igazolja.

`test_predict_topk_returns_highest_first`: A visszaadott valószínűségek csökkenő sorrendben szerepelnek ($\text{probs}[0, i] \geq \text{probs}[0, i + 1]$), vagyis a `topk` valóban a legnagyobb konfidenciájú osztályokat adja vissza, még hozzá rendezetten.

`test_attack_unknown_method_raises`: Hatareset: egy nem létező módszernév ("NEM VALID") megadásakor `ValueError`-t vár, mielőtt bármilyen gradiensszámítás történne – a bemenet-validáció a drága számítás előtt fut le.

`test_attack_fgsm_ignores_iterations_argument`: Annak ellenére, hogy a hívó `iterations=99`-et ad át, FGSM esetén a visszaadott `AttackResult.iterations_run` értéke 1 – igazolva, hogy az FGSM valóban egy lépéses, és a paramétert a diszpécser figyelmen kívül hagyja erre a módszerre.

`test_attack_pgd_reports_requested_iteration_count`: PGD esetén a kért iterációszám (4) változatlanul megjelenik az `iterations_run` mezőben, alátámasztva, hogy a diszpécser a paramétert továbbadja a `_pgd` implementációnak.

`test_attack_fgsm_dispatches_only_to_fgsm:` `mock.patch(..., wraps=_fgsm)` és `wraps=_pgd` spy-okkal figyeli a hívásokat: `method="FGSM"` esetén `_fgsm.assert_called_once()` és `_pgd.assert_not_called()` teljesül. A `wraps` paraméter miatt a valódi implementáció is lefut, tehát az `AttackResult` összeállítása is ellenőrzött marad.

`test_attack_pgd_dispatches_only_to_pgd:` Az előző teszt tükörképe: `method="PGD"` esetén kizárólag `_pgd` hívódik meg, `_fgsm` nem. A két teszt együtt zárja ki, hogy a diszpécser valamelyik ágon „átcsorogva” mindkét algoritmust lefuttassa.

Az FGSM implementáció (`_fgsm`) tesztjei

`test_fgsm_targeted_steps_opposite_direction_of_untargeted:` Azonos bemeneten és ϵ mellett a targeted és non-targeted perturbáció pontosan ellentétes előjelű ($x_{\text{untargeted}} - x = -(x_{\text{targeted}} - x)$), ami közvetlenül a matematikai definícióból (a gradiens előjelének megfordításából) következik.

`test_fgsm_clamps_to_valid_pixel_range:` Szélsőséges bemenet (csupa nulla pixel) és nagy $\epsilon = 0,5$ mellett is a kimenet minden eleme a $[0, 1]$ tartományban marad – igazolva, hogy a `clamp` lépés ténylegesen érvényesül, nem csak elméletben szerepel a specifikációban.

`test_fgsm_result_is_detached_from_autograd:` A visszaadott tenzor `requires_grad` attribútuma `False`.

`test_fgsm_result_has_no_graph_history:` Szigorúbb bizonyíték az előzőnél: `grad_fn` `is None` és `is_leaf` `is True` egyszerre teljesül, ami kizárja azt az esetet is, amikor egy tenzor `requires_grad` értéke hamis, de mégis egy számítási gráf levele marad rejtett hivatkozásokkal.

`test_fgsm_zero_epsilon_returns_original:` Határeset: $\epsilon = 0$ esetén a perturbáció nulla, a kimenet `allclose` az eredeti képpel – az algoritmus degenerált esetben identitásként viselkedik.

`test_fgsm_untargeted_reduces_confidence_in_true_label`: Viselkedési bizonyíték: a `predict()`-en keresztül lekérdezett, a valódi címkéhez tartozó konfidencia a támadás után szigorúan kisebb, mint előtte – ez nem csak a perturbáció létezését, hanem annak tényleges hatását igazolja a modell kimenetére.

`test_fgsm_targeted_increases_confidence_in_target_label`: A `targeted` változat tükörpárja: a mesterségesen kiválasztott célosztály konfidenciája nő a támadás hatására, alátámasztva, hogy a `targeted` mód valóban a célosztály felé tereli a predikciót, nem csupán „valamerre” téveszti meg a modellt.

A PGD implementáció (`_pgd`) tesztjei

`test_pgd_result_stays_within_epsilon_ball`: Mérsékelt lépésméret ($\alpha = 0,02$) mellett a végső perturbáció minden pixelre $|\delta| \leq \epsilon + 10^{-6}$ (a tolerancia a lebegőpontos kerekítést kompenzálja) – ez az ℓ_∞ -gömbbe való projekció alapvető helyességét igazolja.

`test_pgd_hits_ball_when_step_too_large`: Szándékosan irreálisan nagy lépésméretet ($\alpha = 10,0 \gg \epsilon = 0,02$) ad meg, ami minden iterációban túllövést okoz; ilyenkor a projekciós `clamp`-nek kell egyedül biztosítania a korlátot, ezért a tesztben a végső $|\delta|$ pontosan (nem csak $\leq$) ϵ -nak adódik minden pixelen – ez a projekció, nem a lépésméret helyes megválasztása általi korlátozottságot bizonyítja.

`test_pgd_clamps_to_valid_pixel_range`: A pixeltartomány-korlátozás ugyanúgy teljesül PGD esetén is, mint FGSM-nél: szélsőséges (nulla) bemeneten és nagy ϵ mellett a kimenet a $[0, 1]$ tartományban marad.

`test_pgd_result_has_no_graph_history`: Ugyanaz az erős `detach`-bizonyíték (`grad_fn is None` és `is_leaf is True`), mint FGSM esetén – fontos, mert a PGD több iterációt is végez, minden lépésben új `requires_grad_(True)` hívással, így könnyebben „elszivároghatna” egy hivatkozás a gráfra, ha a ciklus végén elmaradna a leválasztás.

`test_pgd_zero_epsilon_returns_original`: Határeset: $\epsilon = 0$ esetén – annak ellenére, hogy több iteráció is lefut – a projekció minden lépésben nulla szélességű intervallumra szorítja a perturbációt, így a végeredmény megegyezik az eredeti képpel.

`test_pgd_untargeted_reduces_confidence_in_true_label`: Viselkedési bizonyíték `predict()`-en keresztül: a valódi címke konfidenciája több iteráció után is csökken – ugyanaz az elv, mint FGSM-nél, de itt azt is igazolja, hogy az iteratív frissítés nem „rontja el” a kumulatív hatást.

`test_pgd_targeted_increases_confidence_in_target_label`: A targeted PGD a célosztály konfidenciáját növeli – ugyanazon tulajdonságpár (non-targeted csökkenti/targeted növeli a releváns konfidenciát) FGSM-re és PGD-re is parametrizált formában lett tesztelve (`@pytest.mark.parametrize`), elkerülve, hogy csak az egyik algoritmusra íródjon le a viselkedés.

4.2. Integrációs tesztek

4.2.1. GUI és támadási motor összekötése

Az integrációs réteg feladata annak ellenőrzése, hogy az `app_interface.py` eseménykezelői helyesen építik-e fel az `AttackContext`-et a ténylegesen betöltött `ModelBundle`-ből, és a `attack_engine.attack()` hívás eredménye helyesen jelenik-e meg a felületen (kép- és predikció-frissítés). Ez a réteg már nem mockolja a modellt, de a valódi felhasználói interakciót (egérkattintás, csúszka-mozgatás) sem szimulálja még – azt a GUI-tesztek rétege végzi.

4.3. Automatizált GUI tesztek

A GUI tesztek a felhasználói felületet magát, a ténylegesen megjelenő ablakot és widgeteket vizsgálják, programozott módon szimulálva a felhasználói interakciókat, majd ellenőrizi, hogy a felület a várt módon reagál-e. Ez a legfelső, egyben legköltségesebb és leglassabb réteg a tesztpiramisban, hiszen minden egyes esethez fel kell építeni a teljes grafikus

felületet – ezért csak kevés, a legfontosabb felhasználói forgatókönyveket lefedő tesztesetekre érdemes szorítkozni, a részletesebb logikai ellenőrzést az alsóbb rétegekre (egység- és integrációs tesztek) hagyva.

4.4. Funkcionális tesztek és felhasználói validáció

A grafikus felület funkcionális, „fekete doboz” teszteseteit a felhasználói forgatókönyvek alapján, *Given–When–Then* formában fogalmazom meg. Az alábbi esetek a tényleges felhasználói felületének elemeihez (gombok, csúszkák, célosztály-kereső) igazodnak, és kézi (manuális) végrehajtásra tervezettek.

- **Kép betöltése a „Load Image” gombbal**

AS A: felhasználó I WANT TO: egy képet betölteni a vizsgálathoz

GIVEN: az alkalmazás elindítva, még nincs kép betöltve

WHEN: a „Load Image” gombra kattint, majd egy érvényes képfájlt választ ki

THEN: az „Original” képmezőben megjelenik a beolvasott, 224×224 -re vágott kép, és alatta a top-3 predikció és konfidencia frissül

- **Modell váltása betöltött kép mellett**

AS A: felhasználó I WANT TO: összehasonlítani, hogy más architektúra hogyan látja a képet

GIVEN: egy kép már be van töltve, az „Original” predikciós panel ki van töltve

WHEN: a modell-választó legördülőben „MobileNetV2”-t választ „ResNet50” helyett

THEN: az „Original” predikciók újraszámolódnak az új modellel; ha korábban már futott támadás, az „Adversarial” predikciók is frissülnek

- **Non-targeted támadás végrehajtása**

AS A: felhasználó I WANT TO: általános tévesztést előidézni a modellben

GIVEN: kép betöltve, „Targeted attack” jelölőnégyzet kikapcsolva, ϵ és iterációs szám csúszkák beállítva

WHEN: az „Attack” gombra kattint

THEN: a státuszsor „Running...” majd „Attack complete” üzenetet mutat, megjelenik a felnagyított perturbáció és a támadott kép, és az „Adversarial” predikciós panelen az eredeti top-1 osztály konfidenciája jellemzően csökken

- **Targeted támadás célosztály kereséssel**

AS A: felhasználó I WANT TO: egy konkrét osztályra rávenni a modellt

GIVEN: kép betöltve, a célosztály-kereső mezőbe elkezd gépelni egy osztálynevet

WHEN: a szűrt listából kiválaszt egy osztályt, majd az „Attack” gombra kattint

THEN: a „Targeted attack” jelölőnégyzet automatikusan bejelölődik, a státuszsor a kiválasztott osztály nevét mutatja, és a támadás után a célosztály konfidenciája nő

- **Epsilon csúszka élő előnézete FGSM módban**

AS A: felhasználó I WANT TO: interaktívan látni, hogyan változik a perturbáció erőssége

GIVEN: FGSM módszerrel már lefutott egy támadás (a gradiens előjele cache-elve van)

WHEN: az ϵ csúszkát új értékre húzza, PGD-re váltás nélkül

THEN: a perturbáció és az adversarial kép új gradiens-számítás *nélkül*, azonnal újraszámolódik a cache-elt előjel alapján

- **Attack gomb kép nélkül**

AS A: felhasználó I WANT TO: (véletlenül) támadást indítani kép betöltése előtt

GIVEN: az alkalmazás elindítva, nincs betöltött kép

WHEN: az „Attack” gombra kattint

THEN: egy „No image” információs párbeszédablak jelenik meg, a támadás nem indul el

- **Védekezési szűrő alkalmazása (Blur / JPEG)**

AS A: felhasználó I WANT TO: megvizsgálni, hogy egy egyszerű szűrő semlegesíti-e a zajt

GIVEN: egy sikeres támadás után az „Adversarial” kép elérhető

WHEN: a „Defend (Blur)” vagy „Defend (JPEG)” gombra kattint

THEN: az „Adversarial” képmező a szűrt képre frissül, és a predikciós panel az újraosztályozott (a szűrés után kapott) eredményeket mutatja

- **Defend gomb adversarial kép nélkül**

AS A: felhasználó I WANT TO: (véletlenül) védekezést alkalmazni támadás előtt

GIVEN: kép be van töltve, de még nem futott támadás

WHEN: valamelyik „Defend” gombra kattint

THEN: egy „No adversarial” információs párbeszédablak jelenik meg, semmilyen kép nem változik

- **Reset gomb**

AS A: felhasználó I WANT TO: új kísérletet kezdeni ugyanazzal a képpel

GIVEN: egy támadás és esetleg egy védekezés már lefutott

WHEN: a „Reset” gombra kattint

THEN: a perturbáció és az „Adversarial” képező üresre vált, az „Adversarial” predikciós panel névtelen („—”) állapotba áll vissza, az „Original” kép és predikciói változatlanok maradnak

- **Hibakezelés támadás közben**

AS A: felhasználó I WANT TO: érthető visszajelzést kapni, ha a támadás valamiért megghiúsul

GIVEN: kép betöltve, a háttérszálon futó `_attack_worker` kivételt dob (pl. eszközhiba miatt)

WHEN: az „Attack” gombra kattint

THEN: egy „Attack failed” hibaablak jelenik meg a teljes veremkiíratással, a státusz sor „Attack failed.” szöveget mutat, az alkalmazás nem fagy le

4.5. Teszteredmények és értelmezésük

4.6. A tesztelés korlátai

5. fejezet

Összefoglalás

5.1. Elért eredmények

5.2. Következtetések

5.3. Továbbfejlesztési lehetőségek

Irodalomjegyzék

- [1] Ian J. Goodfellow, Jonathon Shlens és Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572.
- [2] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras és Adrian Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. 2018. arXiv: 1706.06083.
- [3] Martin Fowler. *TestPyramid*. martinowler.com. 2012. URL: <https://martinfowler.com/bliki/TestPyramid.html>.
- [4] Gintare Karolina Dziugaite, Zoubin Ghahramani és Daniel M. Roy. *A study of the effect of JPG compression on adversarial images*. 2016. arXiv: 1608.00853.